

Generative Models for Physicists

Lei Wang*

Institute of Physics, Chinese Academy of Sciences
Beijing 100190, China

November 5, 2018

Abstract

Generative models generate unseen samples according to a learned joint probability distribution in the high-dimensional space. They find wide applications in density estimation, variational inference, representation learning and more. Deep generative models and associated techniques (such as differentiable programming and representation learning) are cutting-edge technologies physicists can learn from deep learning.

This note introduces the concept and principles of generative modeling, together with applications of modern generative models (autoregressive models, normalizing flows, variational autoencoders etc) as well as the old ones (Boltzmann machines) to physics problems. As a bonus, this note puts some emphasize on physics-inspired generative models which take insights from statistical, quantum, and fluid mechanics.

The latest version of the note is at <http://wangleiphy.github.io/>. Please send comments, suggestions and corrections to the email address in below.

* wanglei@iphy.ac.cn

CONTENTS

1	GENERATIVE MODELING	2
1.1	Probabilistic Generative Modeling	2
1.2	Generative Model Zoo	4
1.2.1	Boltzmann Machines	4
1.2.2	Autoregressive Models	8
1.2.3	Normalizing Flow	9
1.2.4	Variational Autoencoders	12
1.2.5	Tensor Networks	15
1.2.6	Generative Adversarial Networks	17
1.2.7	Generative Moment Matching Networks	17
1.3	Summary	20
2	PHYSICS APPLICATIONS	21
2.1	Variational Ansatz	21
2.2	Renormalization Group	22
2.3	Monte Carlo Update Proposals	22
2.4	Chemical and Material Design	23
2.5	Quantum Information Science and Beyond	24
3	RESOURCES	25
	BIBLIOGRAPHY	26

GENERATIVE MODELING

What I can not create, I do not understand

Richard Feynman's
[blackboard](#)

Deep learning is more than discriminative tasks such as pattern recognition. Generative modeling, by its name, it is about generating new instances from the learned probability distribution. Generative models are fun, useful but also challenging. Thus, they are at the forefront of deep learning research [1].

1.1 PROBABILISTIC GENERATIVE MODELING

The goal of generative modeling is to *represent*, *learn* and *sample* from high-dimensional probability distributions. Given data x and label y , generative models capture the joint probability distribution $p(x, y)$. A well trained generative model can support discriminative tasks through the Bayes formula $p(y|x) = p(x, y) / p(x)$, where $p(x) = \sum_y p(x, y)$. Moreover, one can generate new samples conditioned on its label $p(x|y) = p(x, y) / p(y)$. The generative models are also useful to support semi-supervised learning and reinforcement learning.

For simplicity, let us focus on *density estimation* first, where the goal is to model the joint probability $p(x)$ of a given an unlabelled dataset $\mathcal{D} = \{x\}$. Information theory consideration defines an objective function for this task. The Kullback-Leibler (KL) divergence reads

$$\mathbb{KL}(\pi||p) = \sum_x \pi(x) \ln \left[\frac{\pi(x)}{p(x)} \right], \quad (1)$$

which measures the dissimilarity between two probability distributions. We have $\mathbb{KL} \geq 0$ due to the Jensen inequality. The equality is achieved only when the two distributions match exactly. The KL-divergence is not symmetric with respect to its arguments. So it is not a proper distance measure. $\mathbb{KL}(\pi||p)$ places high probability in p anywhere the data probability π is high, while $\mathbb{KL}(p||\pi)$ places low probability where the data probability π is low [2].

Info

The Jensen inequality [3] states that for convex $\cup$ functions f

$$\langle f(x) \rangle \geq f(\langle x \rangle). \quad (2)$$

Examples of convex $\cup$ functions $f(x) = x^2, e^x, e^{-x}, -\ln x, x \ln x$.

Introducing Shannon entropy $\mathbb{H}(\pi) = -\sum_x \pi(x) \ln \pi(x)$ and cross-entropy $\mathbb{H}(\pi, p) = -\sum_x \pi(x) \ln p(x)$, one sees that $\mathbb{KL}(\pi||p) = \mathbb{H}(\pi, p) - \mathbb{H}(\pi)$. Minimization of the KL-divergence is then equivalent to minimization of the cross-entropy since only it depends on the to-be-optimized parameters. In typical DL applications one only has i.i.d. samples from the target probability distribution $\pi(x)$, so one replaces it with the empirical estimation $\pi(x) = \frac{1}{|\mathcal{D}|} \sum_{x' \in \mathcal{D}} \delta(x - x')$. The cross entropy then turns out to be the negative log-likelihood (NLL) we met in the last chapter

$$\mathcal{L} = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \ln[p(x)]. \quad (3)$$

Minimizing the NLL is a prominent (but not the only) way to train generative models, also known as Maximum Likelihood Estimation (MLE). The Eq. (3) appears to be a minor change compared to the discriminative task. However it causes huge challenges to change the conditional probability to probability function in the cost function. How to represent and learn such high dimensional probability distributions with the intractable normalization factor? How could we marginalize and sample from such high dimensional probability distributions? We will see that physicists have a lot to say about these problems since they love high dimensional probability, Monte Carlo methods and mean-field approaches. In fact, generative modeling has close relation to many problems in statistical and quantum physics, such as inverse statistical problems, modeling a quantum state and quantum state tomography.

Exercise 1 (Positivity of NLL for discrete random variables). Show that the NLL is positive for probability distributions of discrete variables. What about probability densities of continuous variables?

Density estimation on data is not the only thing physicists wanted to do. For example, in statistical physics one wants to solve the problem given bare energy function. Generative models can also help us on that with variational calculation by minimizing the reverse KL-divergence. Consider a statistical physics problem where $\pi(x) = e^{-E(x)} / \mathcal{Z}$ and $\mathcal{Z} = \sum_x e^{-E(x)}$. We try to minimize the free

Idea 1: use a neural net to represent $p(x)$, but how to normalize? how to sample? Idea 2: use a neural net to transform simple prior z to complex data x , but what is the likelihood? How to actually learn the network?

"Solving" means: 1. compute expected value of physical observables, 2. sampling configurations at various temperatures, 3. compute marginal probabilities given arbitrary subgroup of variables

energy $-\ln \mathcal{Z}$, which is unfortunately intractable in general. To proceed, we define a variational free energy

$$\mathcal{L} = \sum_{\mathbf{x}} q(\mathbf{x}) \ln \left[\frac{q(\mathbf{x})}{e^{-E(\mathbf{x})}} \right] = \langle E(\mathbf{x}) + \ln q(\mathbf{x}) \rangle_{\mathbf{x} \sim q(\mathbf{x})} \quad (4)$$

for a normalized variational probability distribution $q(\mathbf{x})$. The two terms have the physical meaning of “energy” and “entropy” respectively. Crucially, since

$$\mathcal{L} + \ln \mathcal{Z} = \mathbb{KL}(q||\pi) \geq 0, \quad (5)$$

thus Eq. (4) is a variational upper bound of the physical free energy, $-\ln \mathcal{Z}$. The approximation becomes exact when the variational distribution approaches to the target distribution. Equation (5) is known as Gibbs-Bogoliubov-Feynman inequality in physics.

The *density estimation* and *variational calculation* examples are by no means the only two applications of the generative models. Since they capture the whole distribution, by defining and adjusting the suitable cost function you can use generative model for creative applications, or enhance performance of downstream tasks. One thing we already see is that the requirement on the generative models are task dependent. For density estimation, we need to efficiently compute the likelihood (or at least its gradient given data). While for variational calculation, we need to be able to efficiently sample from the model as well.

1.2 GENERATIVE MODEL ZOO

This section we review several representative generative models. The key idea is to impose certain structural prior in the probabilistic model. Each model has its own strengths and weakness. Exploring new approaches or combining the existing ones is an active research field in deep learning, with many ideas coming from physics.

1.2.1 Boltzmann Machines

As a prominent statistical physics inspired approach, the Boltzmann Machines (BM) model the probability as a Boltzmann distribution

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\mathcal{Z}}, \quad (6)$$

where $E(\mathbf{x})$ is an energy function and $\mathcal{Z}$, the partition function, is a normalization factor. The task of probabilistic modeling is then translated into designing and learning of the energy function to model observed data. Density estimation of binary data is related to the so

called inverse Ising problem. Exploiting the maximum log-likelihood estimation, the gradient of Eq. (3) is

$$\frac{\partial \mathcal{L}}{\partial \theta} = \left\langle \frac{\partial E(\mathbf{x})}{\partial \theta} \right\rangle_{\mathbf{x} \sim \mathcal{D}} - \left\langle \frac{\partial E(\mathbf{x})}{\partial \theta} \right\rangle_{\mathbf{x} \sim p(\mathbf{x})}. \quad (7)$$

The two terms are called positive and negative phase respectively. Intuitively, the positive phase tries to push down the energy of the observed data, therefore increases the model probability on the observed data. While the negative phase tries to push up the energy on samples drawn from the model, therefore to make the model probability more evenly distributed.

Example

Consider a concrete example of the energy model $E = -\frac{1}{2}W_{ij}x_i x_j$, the gradient Eq. (7) can be simply evaluated. And the gradient descent update

$$W_{ij} = W_{ij} + \eta \left(\langle x_i x_j \rangle_{\mathbf{x} \sim \mathcal{D}} - \langle x_i x_j \rangle_{\mathbf{x} \sim p(\mathbf{x})} \right). \quad (8)$$

The physical meaning of such update is quite appealing: one compares the correlation on the dataset and on the model, then strengthen or weaken the coupling accordingly.

The positive phase are quite straightforward to estimate by simply sampling batches from the dataset. While the negative phase typically involves the Markov chain Monte Carlo (MCMC) sampling. It can be very expensive to thermalize the Markov chain at each gradient evaluation step. The contrastive divergence (CD) algorithm [4] initialize the chain with a sample drawn from the dataset and run the Markov chain only k steps. The reasoning is that if the BM has learned the probability well, then the model probability $p(\mathbf{x})$ resembles the one of the dataset anyway. Furthermore, the persistent CD [5] algorithm use the sample from last step to initialize the Monte Carlo chain. The logic being that the gradient descent update of the model is small anyway, so accumulation of the Monte Carlo samples helps mixing. In practice, one run a batch of the Monte Carlo chains in parallel to estimate the expected value of the negative phase.

Exercise 2 (Mind the Gradient). Define $\Delta = \langle E(\mathbf{x}) \rangle_{\mathbf{x} \sim \mathcal{D}} - \langle E(\mathbf{x}) \rangle_{\mathbf{x} \sim p(\mathbf{x})}$. How is its gradient with respect to θ related to Eq. (7) ?

To increase the representational power of the model, one can introduce hidden variables in the energy function and marginalize them to obtain the model probability distribution

$$p(\mathbf{x}) = \frac{1}{\mathcal{Z}} \sum_{\mathbf{h}} e^{-E(\mathbf{x}, \mathbf{h})}. \quad (9)$$

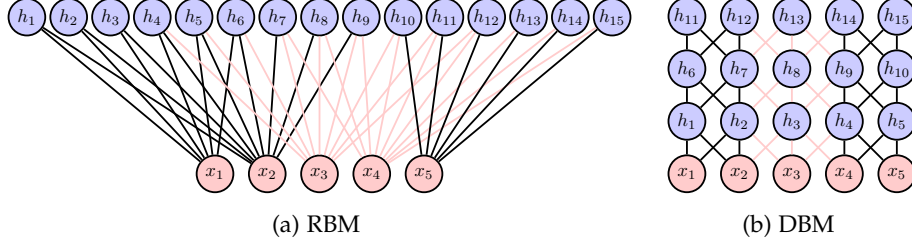


Figure 1: RBM and DBM with the same number of neurons and connections. Information theoretical consideration shows that the DBM can potentially capture patterns that are impossible for the RBM [6].

This is equivalent to say that $E(\mathbf{x}) = -\ln \sum_{\mathbf{h}} e^{-E(\mathbf{x}, \mathbf{h})}$ in Eq. (6), which can be quite complex even for simple joint energy function $E(\mathbf{x}, \mathbf{h})$. Differentiating the equation, we have

$$\frac{\partial E(\mathbf{x})}{\partial \theta} = \frac{\sum_{\mathbf{h}} e^{-E(\mathbf{x}, \mathbf{h})} \frac{\partial E(\mathbf{x}, \mathbf{h})}{\partial \theta}}{\sum_{\mathbf{h}} e^{-E(\mathbf{x}, \mathbf{h})}} = \sum_{\mathbf{h}} p(\mathbf{h}|\mathbf{x}) \frac{\partial E(\mathbf{x}, \mathbf{h})}{\partial \theta}, \quad (10)$$

Therefore, in the presence of the hidden variables the gradient in Eq. (7) becomes

$$\frac{\partial \mathcal{L}}{\partial \theta} = \left\langle \frac{\partial E(\mathbf{x}, \mathbf{h})}{\partial \theta} \right\rangle_{\mathbf{x} \sim \mathcal{D}, \mathbf{h} \sim p(\mathbf{h}|\mathbf{x})} - \left\langle \frac{\partial E(\mathbf{x}, \mathbf{h})}{\partial \theta} \right\rangle_{(\mathbf{x}, \mathbf{h}) \sim p(\mathbf{x}, \mathbf{h})}, \quad (11)$$

which remains simple and elegant. However, the downside of introducing the hidden variables is that one needs even to perform expensive MCMC for the positive phase. An alternative approach is to use the mean-field approximation to evaluate these expectations approximately.

The restricted Boltzmann Machine (RBM) strives to have a balanced expressibility and learnability. The energy function reads

$$E(\mathbf{x}, \mathbf{h}) = -\sum_i a_i x_i - \sum_j b_j h_j - \sum_{i,j} x_i W_{ij} h_j. \quad (12)$$

Since the RBM is defined on a bipartite graph shown in Fig. 1(a), its conditional probability distribution factorizes $p(\mathbf{h}|\mathbf{x}) = \prod_j p(h_j|\mathbf{x})$ and $p(\mathbf{x}|\mathbf{h}) = \prod_i p(x_i|\mathbf{h})$, where

$$p(h_j = 1|\mathbf{x}) = \sigma \left(\sum_i x_i W_{ij} + b_j \right), \quad (13)$$

$$p(x_i = 1|\mathbf{h}) = \sigma \left(\sum_j W_{ij} h_j + a_i \right). \quad (14)$$

This means that given the visible units we can directly sample the hidden units in parallel, vice versa. Sampling back and forth between

Despite of appealing theory and historic importance, BM is now out of fashion in industrial applications due to limitations in its learning and sampling efficiency.

the visible and hidden units is called block Gibbs sampling. Such sampling approach appears to be efficient, but it is not. The visible and hidden features tend to lock to each other for many steps in the sampling. In the end, the block Gibbs sampling is still a form of MCMC which in general suffers from long autocorrelation time and transition between modes.

Info

For an RBM, one can actually trace out the hidden units in the Eq. (6) analytically and obtain

$$E(\mathbf{x}) = - \sum_i a_i x_i - \sum_j \ln(1 + e^{\sum_i x_i W_{ij} + b_j}). \quad (15)$$

This can be viewed as a Boltzmann Machine with fully visible units whose energy function has a softplus interaction. Using Eq. (7) and Eq. (15) one can directly obtain

$$-\frac{\partial \mathcal{L}}{\partial a_i} = \langle x_i \rangle_{\mathbf{x} \sim \mathcal{D}} - \langle x_i \rangle_{\mathbf{x} \sim p(\mathbf{x})}, \quad (16)$$

$$-\frac{\partial \mathcal{L}}{\partial b_j} = \langle p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim \mathcal{D}} - \langle p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim p(\mathbf{x})}, \quad (17)$$

$$-\frac{\partial \mathcal{L}}{\partial W_{ij}} = \langle x_i p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim \mathcal{D}} - \langle x_i p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim p(\mathbf{x})}. \quad (18)$$

One sees that the gradient information is related to the difference between correlations computed on the dataset and the model.

Exercise 3 (Improved Estimators). To reconcile Eq. (11) and Eqs. (16-18), please convince yourself that $\langle x_i p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim \mathcal{D}} = \langle x_i h_j \rangle_{\mathbf{x} \sim \mathcal{D}, \mathbf{h} \sim p(\mathbf{h} | \mathbf{x})}$ and $\langle x_i p(h_j = 1 | \mathbf{x}) \rangle_{\mathbf{x} \sim p(\mathbf{x})} = \langle x_i h_j \rangle_{(\mathbf{x}, \mathbf{h}) \sim p(\mathbf{x}, \mathbf{h})}$. The former ones are improved estimators with reduced variances. In statistics this is known as the Rao-Blackwellization trick. Remember that: whenever you can perform marginalization analytically in a Monte Carlo calculation, please do it.

Although in principle the RBM can represent any probability distribution given sufficiently large number of hidden neurons, the requirement can be exponential. To further increase the representational efficiency, one introduces the deep Boltzmann Machine (DBM) which has more than one layers of hidden neurons, see Fig. 1(b). Under information theoretical considerations, one can indeed show there are certain data which is impossible to represent using an RBM, but can possibly be represented by the DBM with the same number of hidden neurons and connections [6]. However, the downside of DBMs is that they are even harder to train and sample due the interactions among the hidden units [7].

1.2.2 Autoregressive Models

Arguably the simplest probabilistic model is the autoregressive models. They belong to the *fully visible Bayes network*. Basically, they breaks the full probability function into products of conditional probabilities, e.g.,

$$p(\mathbf{x}) = \prod_i p(x_i | \mathbf{x}_{<i}). \quad (19)$$

One can parameterize and learn the conditional probabilities using neural networks. In practice, one can model all these conditional probabilities using a single neural network, either a recurrent neural network with variable length, or using a feedforward neural network with masks. Note that these neural networks do not directly output the sample x_i , but the *parameters* of the conditional probability. For example, for continuous variables we can demand $p(x_i | \mathbf{x}_{<i}) = \mathcal{N}(x_i; \mu_i, \sigma_i^2)$, where the mean and variance are functions of $\mathbf{x}_{<i}$. The log-likelihood of a given datum is easily computed as

$$\ln p(\mathbf{x}) = -\frac{1}{2} \sum_i \left(\left(\frac{x_i - \mu_i}{\sigma_i} \right)^2 + \ln(2\pi\sigma_i) \right). \quad (20)$$

To sample from the autoregressive model, we can sample $\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1})$ and iterate the update rule

$$x_i = \sigma_i(\mathbf{x}_{<i})\epsilon_i + \mu_i(\mathbf{x}_{<i}). \quad (21)$$

Info

A slightly awkward but very enlightening way to compute the log-likelihood of the autoregressive model is to treat Eq. (21) as an invertible mapping between $\mathbf{x}$ and ϵ , and invoke the probability transformation

$$\ln p(\mathbf{x}) = \ln \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1}) - \ln \left| \det \left(\frac{\partial \mathbf{x}}{\partial \epsilon} \right) \right|. \quad (22)$$

Notice that Jacobian matrix is triangular, whose determinant can be easily computed to be Eq. (20). Generalizing this idea to more complex bijective transformations bring us to a general class of generative models called *Normalizing Flow* [8–16]. In particular, a stack of autoregressive transformations is called autoregressive flow (AF).

Despite their simplicity, autoregressive networks have achieved state of the art performances in computer vision (PixelCNN and PixelRNN [13]) and speech synthesis (WaveNet [14]). The downside of

autoregressive models is that one has to impose an order of the conditional dependence which may not correspond to the global hierarchical structure of the data. Moreover, sequential sampling of the autoregressive model such as Eq. (21) is considered to be slow since they can not take advantage of modern hardware. Nevertheless, the generative process Eq. (21) is *direct* sampling, which is can be more efficient compared to the Gibbs sampling of Boltzmann Machines.

Info

The inverse autoregressive flow (IAF) [12] changes the transformation Eq. (21) to be

$$x_i = \sigma_i(\epsilon_{<i})\epsilon_i + \mu_i(\epsilon_{<i}), \quad (23)$$

so that one can generate the data in parallel. The log-likelihood of the generated data also follows Eq. (20). However, the downside of the IAF is that it can not efficiently compute the likelihood of an arbitrary given data which is not generated by itself. Thus, IAF is not suitable for density estimation. IAF was originally introduced to improve the encoder of the VAE [12]. Recently, DeepMind use an IAF (Parallel WaveNet) [17] to learn the probability density of an autoregressive flow (WaveNet) [14], thus to improve the speech synthesis speed to meet the needs in real-world applications [18]. To train the parallel WaveNet, they minimize the *Probability Density Distillation* loss $\mathbb{KL}(p_{\text{IAF}}||p_{\text{AF}})$ [17] since it is easy to draw sample from IAF, and easy to compute likelihood of AF on given data.

1.2.3 Normalizing Flow

Normalizing flow is a family of bijective and differentiable (i.e., diffeomorphism) neural networks which maps between two continuous variables z and x of the same dimension. The idea is that the physical variables can have more complex realistic probability density compared to the latent variables [8–16]

$$\ln p(x) = \ln q(z) - \ln \left| \det \left(\frac{\partial x}{\partial z} \right) \right|. \quad (24)$$

Since diffeomorphism forms a group, the transformation is compositional $x = g(z) = \dots \circ g_2 \circ g_1(z)$, where each step is a diffeomorphism. And the log-Jacobian determinant in Eq. (24) is computed as $\ln \left| \det \left(\frac{\partial x}{\partial z} \right) \right| = \sum_i \ln \left| \det \left(\frac{\partial g_{i+1}}{\partial g_i} \right) \right|$. To compute the log-likelihood of a given data, one first infer $z = g^{-1}(x)$ and keep track of the log-Jacobian determinant in each step.

The abstraction of a diffeomorphism neural network is called a bi-

Bijectors are modular

jector [19, 20]. Each bijector should provide interface to compute forward, inverse and log-Jacobian determinant in an efficient way. The bijectors can be assembled in a modular fashion to perform complex probability transformation. Because of their flexibility, they can act as drop in components of other generative models.

Example

As an example of Eq. (24), consider the famous Box-Muller transformation which maps a pair of uniform random variables z to Gaussian random variables x

$$\begin{cases} x_1 = \sqrt{-2 \ln z_1} \cos(2\pi z_2), \\ x_2 = \sqrt{-2 \ln z_1} \sin(2\pi z_2). \end{cases} \quad (25)$$

Since $\left| \det \left(\frac{\partial x}{\partial z} \right) \right| = \left| \det \begin{pmatrix} \frac{-\cos(2\pi z_2)}{z_1 \sqrt{-2\pi \ln z_1}} & -2\pi \sqrt{-2 \ln z_1} \sin(2\pi z_2) \\ \frac{-\sin(2\pi z_2)}{z_1 \sqrt{-2\pi \ln z_1}} & 2\pi \sqrt{-2 \ln z_1} \cos(2\pi z_2) \end{pmatrix} \right| = \frac{2\pi}{z_1}$, we confirm that $q(x) = p(z) / \left| \det \left(\frac{\partial x}{\partial z} \right) \right| = \frac{1}{2\pi} \exp \left(-\frac{1}{2}(x_1^2 + x_2^2) \right)$. Normalization flows are generalizations of this trick to higher dimensional spaces while still keeping the Jacobian determinants easy to compute.

We take the real-valued non-volume preserving transformation (Real NVP) [11] as an example of the normalizing flow. For each layer of the Real NVP network, we divide multi-dimensional variables x^ℓ into two subgroups $x^\ell = x_{<}^\ell \cup x_{>}^\ell$ and transform one subgroup conditioned on the other group at each step

$$\begin{cases} x_{<}^\ell = x_{<}^\ell \\ x_{>}^\ell = x_{>}^\ell \odot e^{s_\ell(x_{<}^\ell)} + t_\ell(x_{<}^\ell) \end{cases} \quad (26)$$

where $s_\ell(\cdot)$ and $t_\ell(\cdot)$ are two arbitrary functions (with correct input/output dimension) which we parametrize using neural networks. It is clear that this transformation is easy to invert by reversing the scaling and translation operations. Moreover, the Jacobian determinant of the transformation is also easy to compute since the matrix is triangular. By applying a chain of these elementary transformations to various bipartitions one can transform in between a simple prior density and a complex target density. The Real NVP network can be trained with standard maximum likelihood estimation on data. After training, one can generate new samples directly by sampling latent variables according to the prior probability density and passing them through the network. Moreover, one can perform inference by passing the data backward through the network and obtain the latent variables. The log-probability of the data is efficiently computed as

$$\ln q(x) = \ln p(z) - \sum_{\ell, i} (s_\ell)_i, \quad (27)$$

where the summation over index i is for each component of the output of the s function.

From the above discussion, one sees that the crucial point of design flow-based generative model is to balance the expressibility and the computational effort of the Jacobian calculation. Typically, this is implemented via imposing internal structure of the network [8–16]. Recently, References shows that it is possible to take the “continuous-time” limit of the flow, which relax the structure constrain [21, 22]. Interestingly, the resulting generative model can be viewed as integrating an ordinary differential system for the data and its likelihood.

Reference [23] further revealed the optimal transport perspective of continuous-time normalizing flow and discussed means to impose the physical symmetries in the generating process. Consider latent variables z and physical variables x both living in $\mathbb{R}^N$. Given a diffeomorphism between them, $x = x(z)$, the probability densities in the latent and physical spaces are related by $p(z) = q(x) \left| \det \left(\frac{\partial x}{\partial z} \right) \right|$. The Brenier theorem [24] implies that instead of dealing with a multi-variable generative map, one can consider a scalar valued generating function $x = \nabla u(z)$, where the convex Brenier potential u satisfies the Monge-Ampère equation [25]

*Monge-Ampère
Flow*

$$\frac{p(z)}{q(\nabla u(z))} = \det \left(\frac{\partial^2 u}{\partial z_i \partial z_j} \right). \quad (28)$$

Given the densities p and q , solving the Monge-Ampère equation for u turns out to be challenging due to the nonlinearity in the determinant. Moreover, for typical machine learning and statistical physics problems, one faces an additional challenge that one does not even have direct access to both probability densities p and q . Instead, one only has independent and identically distributed samples from one of them, or one only knows one of the distributions up to a normalization constant. Therefore, solving the Brenier potential in these contexts is a control problem instead of a boundary value problem. An additional computational challenge is that even for a given Brenier potential, the right-hand side of (28) involves the determinant of the Hessian matrix, which scales unfavorably as $\mathcal{O}(N^3)$ with the dimensionality of the problem.

To address these problems, we consider the Monge-Ampère equation in its *linearized form*, where the transformation is infinitesimal [26]. We write the convex Brenier potential as $u(z) = ||z||^2/2 + \epsilon \varphi(z)$, thus $x - z = \epsilon \nabla \varphi(z)$, and correspondingly $\ln q(x) - \ln p(z) = -\text{Tr} \ln \left(I + \epsilon \frac{\partial^2 \varphi}{\partial z_i \partial z_j} \right) = -\epsilon \nabla^2 \varphi(z) + \mathcal{O}(\epsilon^2)$. In the last equation we expand the logarithmic

function and write the trace of a Hessian matrix as the Laplacian operator. Finally, taking the continuous-time limit $\epsilon \rightarrow 0$, we obtain

$$\frac{dx}{dt} = \nabla \varphi(x), \quad (29)$$

$$\frac{d \ln p(x, t)}{dt} = -\nabla^2 \varphi(x), \quad (30)$$

such that $x(0) = z$, $p(x, 0) = p(z)$, and $p(x, T) = q(x)$, where t denotes continuous-time and T is a fixed time horizon. For simplicity here, we still keep the notion of x , which now depends on time. The evolution of x from time $t = 0$ to T then defines our generative map.

The two ordinary differential equations (ODEs) compose a dynamical system, which describes the flow of continuous random variables and the associated probability densities under iterative change-of-variable transformation. One can interpret equations (29) and (30) as fluid mechanics equations in the Lagrangian formalism. Equation (29) describes the trajectory of fluid parcels under the velocity field given by the gradient of the potential function $\varphi(x)$. While the time derivative in (30) is understood as the “material derivative” [27], which describes the change of the local fluid density $p(x, t)$ experienced by someone travels with the fluid.

The fluid mechanics interpretation is even more apparent if we write out the material derivative in (30) as $d/dt = \partial/\partial t + dx/dt \cdot \nabla$, and use (29) to obtain

$$\frac{\partial p(x, t)}{\partial t} + \nabla \cdot [p(x, t)v] = 0, \quad (31)$$

which is the continuity equation of a *compressible fluid* with density $p(x, t)$ and velocity field $v = \nabla \varphi(x)$. Obeying the continuity equation ensures that the flow conserves the total probability mass. Moreover, the velocity field is curl free $\nabla \times v \equiv 0$ and the fluid follows a form of *gradient flow* [28]. The irrotational flow matches one’s heuristic expectation that the flow-based generative model transports probability masses.

It should be stressed that although we use the optimal transport theory to motivate model architecture design, i.e., the gradient flow for generative modeling, we do not have to employ the optimal transport objective functions for the control problem. The difference is that in generative modeling one typically fixes only one end of the probability density and aims at learning a suitable transformation to reach the other end. While for optimal transport one has both ends fixed and aims at minimizing the transportation cost. References [29–31] adapted the Wasserstein distances in the optimal transport theory as an objective function for generative modeling.

1.2.4 Variational Autoencoders

Variational autoencoder (VAE) is an elegant framework for per-

One of the creators of VAE, Max Welling, did his PhD on gravity theory under the supervision of ‘t Hooft in late 90s.

forming variational inference [32], which also has deep connection variational mean field approaches in statistical physics. In fact, the predecessor of VAE is called Helmholtz machines [33]. The general idea of an autoencoder is to let the input data go through a network with bottleneck and restore itself. After training, the first half of the network is an encoder which transform the data x into the latent space z . And the second half of the network is a decoder which transform latent variables into the data manifold. The bottleneck means that we typically require that the latent space has lower dimension or simpler probability distribution than the original data.

Suppose the latent variables $p(z)$ follow a simple prior distribution, such as an independent Gaussian. The decoder is parameterized by a neural network which gives the conditional probability $p(x|z)$. Thus, the joint probability distribution of the visible and latent variables is also known $p(x, z) = p(x|z)p(z)$. However, the encoder probability given by the posterior $p(z|x) = p(x, z)/p(x)$ is much more difficult to evaluate since normalization factor $p(x)$ is intractable. One needs to marginalize the latent variables z in the joint probability distribution $p(x) = \int p(x, z)dz$.

The intractable integration over the latent variables also prevent us minimizing the NLL on the dataset. To deal with such problem, we employ variational approach originated from statistical physics. The variational Bayes methods is a an application of the variational free energy calculation in statistical physics Eq. (5) for inference problem. For each data we introduce

$$\mathcal{L}(x) = \langle -\ln p(x, z) + \ln q(z|x) \rangle_{z \sim q(z|x)}, \quad (32)$$

which is a variational upper bound of $-\ln p(x)$ since $\mathcal{L}(x) + \ln p(x) = \mathbb{KL}(q(z|x)||p(z|x)) \geq 0$. We see that $q(z|x)$ provides a variational approximation of the posterior $p(z|x)$. By minimizing $\mathcal{L}$ one effectively pushes the two distributions together. And the variational free energy becomes exact only when $q(z|x)$ matches to $p(z|x)$. In fact, $-\mathcal{L}$ is called evidence lower bound (ELBO) in variational inference.

We can obtain an alternative form of the variational free energy

$$\mathcal{L}_{\theta, \phi}(x) = -\langle \ln p_{\theta}(x|z) \rangle_{z \sim q_{\phi}(z|x)} + \mathbb{KL}(q_{\phi}(z|x)||p(z)). \quad (33)$$

The first term of Eq. (33) is the reconstruction negative log-likelihood, while the second term is the KL divergence between the approximate posterior distribution and the latent prior. We also be explicit about the network parameters θ, ϕ of the encoder and decoder.

The decoder neural network $p_{\theta}(x|z)$ accepts the latent vector z and outputs the parametrization of the conditional probability. It can be

$$\ln p_{\theta}(x|z) = \sum_i x_i \ln \hat{x}_i + (1 - x_i) \ln(1 - \hat{x}_i), \quad (34)$$

$$\hat{x} = \text{DecoderNeuralNet}_{\theta}(z), \quad (35)$$

Intractable posterior

This breakup is also the foundation of the Expectation-Maximization algorithm, where one iterates alternatively between optimizing the variational posterior (E) and the parameters (M) to learn models with latent variables [34].

for binary data. And

$$\ln p_{\theta}(x|z) = \ln \mathcal{N}(x; \mu, \sigma^2 \mathbf{1}), \quad (36)$$

$$(\mu, \sigma) = \text{DecoderNeuralNet}_{\theta}(z), \quad (37)$$

for continuous data. Gradient of Eq. (33) with respect to θ only depends on the first term.

Similarly, the encoder $q_{\phi}(z|x)$ is also parametrized as a neural network. To optimize ϕ we need to compute the gradient with respect to the sampling process, which we invoke the *reparametrization trick*. To generate sample $z \sim q_{\phi}(z|x)$ we first sample from an independent random source, say $\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1})$ and pass it through an invertible and differentiable transformation $z = g_{\phi}(x, \epsilon)$. The probability distribution of the encoder is related to the one of the random source by

$$\ln q_{\phi}(z|x) = \ln \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1}) - \ln \left| \det \left(\frac{\partial g_{\phi}(x, \epsilon)}{\partial \epsilon} \right) \right|. \quad (38)$$

Suppose that the log-determinant is easy to compute so we can sample the latent vector z given the visible variable x and an independent random source ϵ . Now that the gradient can easily pass through the sampling process

$$\nabla_{\phi} \langle f(x, z) \rangle_{z \sim q_{\phi}(z|x)} = \langle \nabla_{\phi} f(x, g_{\phi}(x, \epsilon)) \rangle_{\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1})}. \quad (39)$$

Info

As an alternate, the REINFORCE [35] (score function) estimator of the gradient reads

$$\nabla_{\phi} \langle f(x, z) \rangle_{z \sim q_{\phi}(z|x)} = \langle f(x, z) \nabla_{\phi} \ln q_{\phi}(z|x) \rangle_{z \sim q_{\phi}(z|x)}. \quad (40)$$

Compared to the reparametrization Eq. (39) REINFORCE usually has larger variance because it only uses the scalar function $\ln q_{\phi}(z|x)$ instead of the vector information of the gradient $\nabla_{\phi} f(x, z)$. An advantage of REINFORCE is that it can also work with discrete latent variables. See Ref. [36] for the research frontier for low variance unbiased gradient estimation for discrete latent variables.

Suppose each component of the latent vector follows independent Gaussian whose mean and variance are determined by the data x , we have

$$\ln q_{\phi}(z|x) = \ln \mathcal{N}(z; \mu, \sigma^2 \mathbf{1}), \quad (41)$$

$$(\mu, \sigma) = \text{EncoderNeuralNet}_{\phi}(x). \quad (42)$$

And the way to sample the latent variable is

$$\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{1}), \quad (43)$$

$$z = \mu + \sigma \odot \epsilon. \quad (44)$$

The KL term in Eq. (33) can be evaluated analytically [32] in this case.

After training of the VAE, we obtain an encoder $q(z|x)$ and a decoder $p(x|z)$. The encoder performs dimensionality reduction from the physical space into the latent space. Very often, different dimensions in the latent space acquire semantic meaning. By performing arithmetic operations in the latent space one can interpolate between physical data. Optimization of chemical properties can also be done in the low dimensional continuous latent space. The decoder is a generative model, which maps latent variable into the physical variable with rich distribution.

Info

The marginal NLL of the VAE can be estimated using importance sampling

$$-\ln p(x) = -\ln \left\langle \frac{p(x, z)}{q(z|x)} \right\rangle_{z \sim q(z|x)}. \quad (45)$$

By using the Jensen's inequality (2) one can also see that the variational free energy Eq. (32) is an upper bound of Eq. (45).

1.2.5 Tensor Networks

A new addition to the family of generative models is the tensor network state. In a quantum inspired approach one models the probability as the wavefunction square

$$p(x) = \frac{|\Psi(x)|^2}{\mathcal{Z}}, \quad (46)$$

where $\mathcal{Z}$ is the normalization factor. This representation, named as Born Machine [37], transforms many approaches of representing quantum state into machine learning. Consider binary data, we can represent wavefunction using the matrix product state (MPS) [38]

$$\Psi(x) = \text{Tr} \left(\prod_i A^i[x_i] \right). \quad (47)$$

The size of each matrix is called the bond dimension of the MPS representation. They control the expressibility of the MPS parameterization. The MPS can be learned using maximum likelihood estimation as before. Although other loss functions such as fidelity of quantum states can also be considered [39, 40].

An advantage of using MPS for generative modeling is that one can adopt algorithms developed for quantum many-body states such as the DMRG for parameter learning. For example, one can perform "two-site" optimization by merging two adjacent matrices together and optimizing its tensor elements. After the optimization the rank

Adaptive learning

of the two site tensor may grow, one can thus dynamically adjust the bond dimension of the MPS representation during learning. As a consequence, the expressibility of the model grows as it observes the data, which is different from conventional generative models with fixed network with fixed number of parameters.

Another advantage of MPS as a generative model is that the gradient of the NLL (3) can be computed efficiently

Efficient gradient

$$\frac{\partial \mathcal{L}}{\partial \theta} = -2 \left\langle \frac{\partial \ln \Psi(\mathbf{x})}{\partial \theta} \right\rangle_{\mathbf{x} \sim \mathcal{D}} + 2 \left\langle \frac{\partial \ln \Psi(\mathbf{x})}{\partial \theta} \right\rangle_{\mathbf{x} \sim p(\mathbf{x})}. \quad (48)$$

Note that the negative phase (second term) can also be written as $\mathcal{Z}' / \mathcal{Z}$, where $\mathcal{Z}' = 2 \sum_{\mathbf{x}} \Psi'(\mathbf{x}) \Psi(\mathbf{x})$ and the prime means derivatives with respect to the network parameter θ . Crucially, for MPS both $\mathcal{Z}'$ and $\mathcal{Z}$ can be evaluated efficiently via tensor contractions. So the gradient can be computed efficiently without resorting to the contrastive divergence, in contrast to the Boltzmann Machines (7). The NLL is also tractable so that MPS model knows the normalized density of each sample.

Finally, tractable normalization factor of MPS allows one to perform *direct* sampling instead of using MCMC used in the Boltzmann Machines. While compared to the autoregressive models, one can perform data restoration by removing any part of the data. This is because tensor networks expresses an undirected (instead of directed) probability dependence for the data.

Direct sampling

These aforementioned advantages apply as well to other unitary tensor networks such as the tree tensor network and MERA. It is yet to be seen whether one can unlock the potential of tensor networks for real world AI applications. Using Eq. (46) and associated quantum-inspired approaches (or even a quantum device) provide a great chance to model complex probabilities. While on a more conceptual level, one wish to have more quantitative and interpretable approaches inspired by quantum physics research. For example, Born Machine may give us more principled structure designing and learning strategies for modeling complex dataset, and provide a novel theoretical understandings of the expressibility of generative models the quantum information perspective.

In a more general context, there are at least two reasons of using the tensor networks for machine learning. First, tensor network and algorithms provide principled approaches for discriminative and generative tasks with possibly stronger representational power. In fact, the mathematical structure of tensor network states appear naturally when one tries to extend the probabilistic graphical models while still attempt to ensure the positivity of the probability density [41, 42]. Second, tensor networks are doorways to quantum machine learning because many of the tensor networks are formally equivalent to quantum circuits. Tensor network states provide means of architecture design and parameter initialization for quantum circuits [43]. By

now, tensor networks have certainly caught attentions of some of the machine learning practitioners [44, 45]. However, we are still awaiting for an event similar to AlexNet, where the tensor network machine learning approach wins over the traditional approaches by a large margin. With accumulations of the results and techniques, this is likely to happen in the coming years, at least in one specific application domain.

A final note, besides various tensor network *architectures*, there are large number of *algorithms* for optimizing, evolving, factorizing, and compressing tensor networks. Beside simulating quantum many-body physics, they have far reaching impacts in applications ranging from classical graphical models all the way to quantum computing. Please refer to this [Website](#) for more information.

1.2.6 Generative Adversarial Networks

Different from the generative models introduced till now, the Generative Adversarial Networks (GAN) belong to the *implicit* generative models [46].

That is to say that although one can generate samples using GAN, one does not have direct access to its likelihood. So obviously training of GAN is also not based on maximum likelihood estimation.

A generator network maps random variables z to physical data x . A discriminator network D is a binary classifier which tries tell whether the sample is from the dataset $\mathcal{D}$ (1) or synthesized (0). On the expanded dataset $\{(x, 1), (G(z), 0)\}$, the cross-entropy cost reads

$$\mathcal{L} = -\langle \ln D(x) \rangle_{x \sim \mathcal{D}} - \langle \ln (1 - D(G(z))) \rangle_{z \sim p(z)}. \quad (49)$$

Such cost function defines a minimax game $\max_G \min_D \mathcal{L}$ between the generator and the discriminator, where the generator tries to forge data to confuse the discriminator.

Since the loss function does not involve the probability of the generated samples, one can use an arbitrary neural network as the generator. Giving up likelihood increases the flexibility of the generator network at the cost that it is harder to train and evaluate. Assess the performance of GAN in practice often boils down to beauty contest. Lacking an explicit likelihood function also limits its applications to physics problems where quantitative results are important.

1.2.7 Generative Moment Matching Networks

The generative moment matching networks (GMMN) [47, 48] is another class of implicit generative model which employs the kernel two-sample test as the cost function [49].

The idea is to simply compare the distance in the kernel feature space on the samples drawn from the target and the model distribu-

tions. We refer the following loss function as the squared maximum mean discrepancy (MMD) [50, 51]

$$\begin{aligned}\mathcal{L} &= \left\| \sum_x p(x)\phi(x) - \sum_x \pi(x)\phi(x) \right\|^2 \\ &= \langle K(x, y) \rangle_{x \sim p, y \sim p} - 2\langle K(x, y) \rangle_{x \sim p, y \sim \pi} + \langle K(x, y) \rangle_{x \sim \pi, y \sim \pi}.\end{aligned}\tag{50}$$

The summation in the first line runs over the whole Hilbert space. The expectation values in the second line are for the corresponding probability distributions. The function ϕ maps x to a high-dimensional reproducing kernel Hilbert space [52]. However, as common in the kernel methods, by defining a kernel function $K(x, y) = \phi(x)^T \phi(y)$ one can avoid working in the high-dimensional feature space. We employ a mixture of Gaussians kernel $K(x, y) = \frac{1}{c} \sum_{i=1}^c \exp\left(-\frac{1}{2\sigma_i} |x - y|^2\right)$ to reveal differences between the two distributions under various scales. Here, σ_i is the bandwidth parameter which controls the width of the Gaussian kernel. The MMD loss with Gaussian kernels asymptotically approaches zero if and only if the output distribution matches the target distribution exactly [50, 51].

Table 1: A summary of generative models and their salient features. Question marks mean generalizations are possible, but nontrivial.

Name	Training Cost	Data Space	Latent Space	Architecture	Sampling	Likelihood	Expressibility	Difficulty (Learn/Sample)
RBM	Log-likelihood	Arbitrary	Arbitrary	Bipartite	MCMC	Intractable partition function	★	☠☠☠/☠☠☠☠
DBM	ELBO	Arbitrary	Arbitrary	Bipartite	MCMC	Intractable partition function & posterior	★★★★	☠☠☠/☠☠☠☠
Autoregressive Model	Log-likelihood	Arbitrary	None	Ordering	Sequential	Tractable	★★	☠/☠☠
Normalizing Flow	Log-likelihood	Continuous	Continuous, Same dimension as data	Bijector	Parallel	Tractable	★★	☠/☠
VAE	ELBO	Arbitrary	Continuous	Arbitrary?	Parallel	Intractable posterior	★★★★	☠/☠
MPS/TTN	Log-likelihood	Arbitrary?	None or tree tensor	No loop	Sequential	Tractable	★★★★	☠☠☠/☠☠☠
GAN	Adversarial	Continuous	Arbitrary?	Arbitrary	Parallel	Implicit	★★★★★	☠☠☠☠☠/☠☠☠☠☠
Quantum Circuit	Adversarial	Discrete	Discrete	Arbitrary	Parallel	Implicit	★★★★★★	☠☠☠☠☠☠/☠☠☠☠☠☠

1.3 SUMMARY

In the discussions of generative models we have touched upon a field called probabilistic graphical models [53]. They represent dependence between random variables using graphical notations. The graphical models with undirected edges are called Markov random field, which can be understood as statistical physics models (Sec. 1.2.1). Typically, it is hard to sample from a Markov random field model unless it has a tree structure. (Or, in special cases such as planar Ising model in the absence of magnetic field.) While the graphical models with directed edges are called Bayes network, which describe conditional probability distribution (Sec. 1.2.2). The conditional probabilities allows ancestral sampling starting from the root node and follow the conditional probabilities.

As we have seen, feedforward neural networks can be used as key components for generative modeling. They transform probability distribution of input data to certain target probability distribution. Please note that there are subtle differences in the interpretations of these neural nets' outputs. They can either be parametrization of the conditional probability $p(x|z)$ (Secs. 1.2.2, 1.2.4) or be the samples x themselves (Secs. 1.2.3, 1.2.6). Table 1 summarized and compared the main features of various generative models discussed in this note.

No surprisingly, various generative models are related in various ways. Revealing their connections and seeking for a unified framework calls for a deeper understanding of generative modeling. First of all, the Boltzmann Machines, and in general all probabilistic graphical models, are likely to be closely related to the tensor networks. In particular cases, the exact mappings between RBM and tensor networks has been worked out [6]. It is still rewarding to explore more connections of representation and learning algorithms between these two classes of models. Second, the autoregressive models for continuous variables are closely related to normalizing flows. While Ref. [8] also discussed connection of normalizing flows to the variational autoencoders. Finally, combining models to take advantage of both worlds is also a rewarding research direction [10, 12, 54, 55].

PHYSICS APPLICATIONS

We arrange the following topics according to their application domains.

2.1 VARIATIONAL ANSATZ

Reference [56] obtained excellent variational energy for non-frustrated quantum spin systems by adopting the Restricted Boltzmann Machines in Sec. 1.2.1 as a variational ansatz. The ansatz can be viewed as a generalization of Jastrow trial wavefunctions. The RBM is more flexible in the sense that it encodes multi-body correlations in an efficient way [57]. Moreover, employing complex parametrization extends the ability of RBM, so it may also capture quantum states of frustrated systems and unitary dynamics of quantum circuits [56, 58].

References [6, 59, 60] connect the RBM variational ansatz to tensor network states. References [61, 62] analyzed their expressibility from quantum entanglement and computational complexity points of view respectively. Out of these works, one sees that the neural network states can be advantageous for describing highly entangled quantum states and models with long range interactions. An interesting application is on the chiral topological states, in which the standard PEPS ansatz suffers from fundamental difficulties [60, 63].

Another interesting direction is based on the fact that the RBM, in particular, the one used in [56] is equivalent to a shallow convolutional neural networks. Along this line, it is natural to go systematically to deeper neural networks and employ deep learning frameworks for automatic differentiation in the VMC calculation [64]. Reference [65] performed variational calculation of excited states using backprop optimization of deep neural networks. Reference [66] carried out the VMC calculation for small molecules in the first quantization formalism, in which the antisymmetric property of the wavefunction was taken with special care.

It appears to the author that further development calls for innovations in the optimization scheme, which goes beyond the wavefunction ansatz, e.g. direct generative sampling, and low variance gradient estimator. As a one step towards this goal, the authors employ variational autoregressive networks to study Ising and spin glasses problems [67]. This calculation follows the variational free

energy framework of Eq. (5), which generalizes the celebrated Weiss and Bethe mean field theories.

2.2 RENORMALIZATION GROUP

Renormalization Group (RG) is a fundamental concept in theoretical physics. In essence, RG keeps relevant information while reducing the dimensionality of data. This strongly resembles the quest for representation learning in generative modeling. The connection of RG and deep learning is both intriguing and rewarding. On one hand side it may provide theoretical understanding to deep learning. And on the other hand, it brings deep learning machineries into solving physical problems with RG.

References [68] proposed a generative Bayesian network with a MERA inspired structure. Reference [69] connects the Boltzmann Machines with decimation transformation in real-space RG. Reference [70] connects principal component analysis with momentum shell RG. Reference [71] proposed to use mutual information as a criteria for restoring the RG behavior in the training of Boltzmann Machines. Lastly, Reference [72] proposed a variational RG framework by stacking the bijectors (Sec. 1.2.3) into a MERA-like structure. The approach provides a way to identify collective variables and their effective interaction. The collective variables in the latent space has reduced mutual information. They can be regarded as nonlinear and adaptive generalizations of wavelets. Training of the NeuralRG network employs the probability density distillation (Sec. 1.2.2) on the bare energy function, in which the training loss provides a variational upper bound of the physical free energy. The NeuralRG approach implements an information preserving RG procedure, which is useful for exploring holographic duality [73].

2.3 MONTE CARLO UPDATE PROPOSALS

Markov chain Monte Carlo (MCMC) finds wide applications in physics and machine learning. Since the major drawback of MCMC compared to other approximate methods is its efficiency, there is a strong motivation to accelerate MCMC simulations within both physics and machine learning community. Loosely speaking, there are at least three ideas to accelerate Monte Carlo sampling using machine learning techniques.

First, Reference [74] trained surrogate functions to speed up hybrid Monte Carlo simulation [75] for Bayesian statistics. The surrogate function approximates the potential energy surface of the target problem and provides an easy way to compute derivatives. Recently, there were papers reporting similar ideas for physics problems. Here, the surrogate model can be physical models such as the Ising model [76]

or molecular gases [77], or general probabilistic graphical models such as the restricted Boltzmann machine [78]. For Monte Carlo simulations involving fermion determinants [77, 79] the approach is more profitable since the updates of the original model is much heavier than the surrogate model. However, the actual benefit depends on the particular problem and the surrogate model. A drawback of these surrogate function approaches is that they require training data to start with, which is known as the "cold start" problem in analog to the recommender systems [77]. Using the adaptive approach of [80] one may somewhat alleviate this "chicken-egg" problem.

Second, there were more recent attempts in machine learning community trying to directly optimize the proposal probability via reinforcement learning [81–83]. These papers directly parameterize the proposal probability as neural networks and optimize objective functions related to the efficiency, e.g., the difference of proposals such as the squared jump distances. To ensure unbiased physical results, it is crucial to keep track of the proposal probability of an update and its reverse move for the detailed balance condition. Both A-NICE-MC [81] and L2HMC [82] adopted normalizing flows (Sec. 1.2.3). The later paper is particularly interesting because it systematically generalizes the celebrated hybrid Monte Carlo [75] to a learnable framework. Reference [84] used reinforcement learning to devise updates for frustrated spin models. In fact, besides the efficiency boost one can aim at algorithmic innovations in the Monte Carlo updates. Devising novel update strategies which can reduce the auto correlation between samples was considered to be the art MCMC methods. An attempt along this line is Ref. [85], which connected the Swendsen-Wang cluster and the Boltzmann Machines and explored a few new cluster updates.

Lastly, the NeuralRG technique [72] provides another approach to learn Monte Carlo proposal without data. Since the mapping to the latent space reduces the complexity of the distribution, one can perform highly efficient (hybrid) Monte Carlo updates in the latent space and obtain unbiased physical results. This can be regarded as an extension of the Fourier space or wavelets basis Monte Carlo methods, except that now the representation is learned in an adaptive fashion.

By the way, to obtain unbiased physical results one typically ensures detailed balance condition using Metropolis-Hastings acceptance rule. Thus, one should employ generative models with *explicit* and *tractable* densities for update proposals. This rules out GAN and VAE in the game, at least for the moment.

2.4 CHEMICAL AND MATERIAL DESIGN

Reference [86] use the VAE to map string representation of molecules (SMILES) to a continuous latent space and then perform differen-

liable optimization for desired molecular properties. Like many deep learning applications in natural language and image processing, the model learned meaningful low dimensional representation in the latent space. Arithmetics operations have physical (or rather chemical) meanings. There were also attempts of using GANs for chemistry and material design. See [87] for a recent review.

2.5 QUANTUM INFORMATION SCIENCE AND BEYOND

Generative models were used for quantum error correction [88], for quantum state tomography [89, 90], and for detecting Bell nonlocality in quantum mechanics [91]. Early works mostly employed the restricted Boltzmann Machines. However more recently researches started to explore general class of generative models [90].

Conversely, generative modeling might be one of the killer application of the quantum machine learning [92, 93]. Building on the probabilistic interpretation of quantum mechanics, one can envision a quantum generative model [94]. It expresses the probability distribution of a dataset as the probability associated with the wavefunction. The theory behind it is that even measured on a fixed bases, the quantum circuit can express probability distribution that are intractable to classical computers. In particular, there was a demonstration [95] on training quantum circuit as a probabilistic generative model.

A quantum circuit implementation of the "Born Machine" would be an implicit generative model since one usually does not have direct access to the quantum state of an actual quantum state. The moment matching (Sec. 1.2.7) or adversarial training (Sec. 1.2.6) against a classical neural network can be a way to learn the quantum circuit Born machine [96, 97]. References [98] explores the practical aspects of scaling up the generative learning on actual quantum devices and gradient based training in this setting.

RESOURCES

Tutorial and Reviews

- [Ian Goodfellow, NIPS 2016 Tutorial: Generative Adversarial Networks, Video](#)
- [Ruslan Salakhutdinov, Learning Deep Generative Models](#)
- [Shakir Mohamed and Danilo Rezende, Deep Generative Models, Video](#)
- [Ryan Adams, A Tutorial on Deep Probabilistic Generative Models](#)
- [David Duvenaud, Generative Models, Video](#)

Blogs

- [Rui Shu, Change of Variables: A Precursor to Normalizing Flow](#)
- [Eric Jang, Normalizing Flows Tutorial I, II](#)
- [OpenAI, Glow: Better Reversible Generative Models](#)
- [DeepMind, WaveNet: A Generative Model for Raw Audio, High-fidelity speech synthesis with WaveNet](#)

Codes

- [MADE \(Masked Autoencoder Density Estimation\)](#)
- [PixelCNN](#)
- [RBM for image restoration](#)
- [NetKet](#)
- [Variational Autoregressive Network](#)
- [Neural Renormalization Group \(with Real NVP layers\)](#)
- [Continuous-time Normalizing Flow](#)
- [MPS for generative modeling](#)

ACKNOWLEDGEMENTS

We thank Giuseppe Carleo, Jing Chen, Song Cheng, Shuo-Hui Li, Jinguo Liu, Linfeng Zhang, Pan Zhang and Yueshui Zhang for discussions and shaping perspectives presented in this tutorial. The author is supported by the National Natural Science Foundation of China under Grant No. 11774398.

BIBLIOGRAPHY

- [1] Andrej Karpathy, Pieter Abbeel, Greg Brockman, Peter Chen, Vicki Cheung, Rocky Duan, Ian GoodFellow, Durk Kingma, Jonathan Ho, Rein Houthooft, Tim Salimans, John Schulman, Ilya Sutskever, and Wojciech Zaremba. OpenAI Post on Generative Models, 2016. URL <https://blog.openai.com/generative-models/>.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL <http://www.deeplearningbook.org/>.
- [3] David J C MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [4] Geoffrey E Hinton. Training Products of Experts by Minimizing Contrastive Divergence. *Neural Comput.*, 14:1771, 2002.
- [5] Tijmen Tieleman. Training Restricted Boltzmann Machines using Approximations to the Likelihood Gradient. In *Proceedings of the 25th international conference on Machine Learning*, page 1064, 2008.
- [6] Jing Chen, Song Cheng, Haidong Xie, Lei Wang, and Tao Xiang. On the Equivalence of Restricted Boltzmann Machines and Tensor Network States. *Phys. Rev. B*, 97:085104, 2018. doi: 10.1103/PhysRevB.97.085104. URL <http://arxiv.org/abs/1701.04831>.
- [7] Ruslan Salakhutdinov. Learning Deep Generative Models. *Annu. Rev. Stat. Its Appl.*, 2:361–385, 2015. doi: 10.1146/annurev-statistics-010814-020120. URL <http://www.annualreviews.org/doi/10.1146/annurev-statistics-010814-020120>.
- [8] Laurent Dinh, David Krueger, and Yoshua Bengio. NICE: Non-linear Independent Components Estimation. *arXiv*, 2014. doi: 1410.8516. URL <http://arxiv.org/abs/1410.8516>.
- [9] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. MADE: Masked Autoencoder for Distribution Estimation. *arXiv*, 2015. URL <http://arxiv.org/abs/1502.03509>.
- [10] Danilo Jimenez Rezende and Shakir Mohamed. Variational Inference with Normalizing Flows. *arXiv*, 2015. URL <http://arxiv.org/abs/1505.05770>.

- [11] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. *arXiv*, 2016. doi: 1605.08803. URL <http://arxiv.org/abs/1605.08803>.
- [12] Diederik P. Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improving Variational Inference with Inverse Autoregressive Flow. *arXiv*, 2016. URL <http://arxiv.org/abs/1606.04934>.
- [13] Aaron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel Recurrent Neural Networks. In *Int. Conf. Mach. Learn. I(CML)*, volume 48, pages 1747—1756, 2016. URL <https://arxiv.org/pdf/1601.06759.pdf>
<http://arxiv.org/abs/1601.06759>.
- [14] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A Generative Model for Raw Audio. *arXiv*, 2016. URL <http://arxiv.org/abs/1609.03499>.
- [15] George Papamakarios, Theo Pavlakou, and Iain Murray. Masked Autoregressive Flow for Density Estimation. *arXiv*, 2017. URL <http://arxiv.org/abs/1705.07057>.
- [16] D. P. Kingma and P. Dhariwal. Glow: Generative Flow with Invertible 1x1 Convolutions. *ArXiv e-prints*, July 2018.
- [17] Aaron van den Oord, Yazhe Li, Igor Babuschkin, Karen Simonyan, Oriol Vinyals, Koray Kavukcuoglu, George van den Driessche, Edward Lockhart, Luis C. Cobo, Florian Stimberg, Norman Casagrande, Dominik Grewe, Seb Noury, Sander Dieleman, Erich Elsen, Nal Kalchbrenner, Heiga Zen, Alex Graves, Helen King, Tom Walters, Dan Belov, and Demis Hassabis. Parallel WaveNet: Fast High-Fidelity Speech Synthesis. *arXiv*, 2017. URL <http://arxiv.org/abs/1711.10433>.
- [18] Aaron van den Oord, Yazhe Li, and Igor Babuschkin. High-fidelity speech synthesis with WaveNet, 2017. URL <https://deepmind.com/blog/high-fidelity-speech-synthesis-wavenet/>.
- [19] Pyro Developers. Pyro, 2017. URL <http://pyro.ai/>.
- [20] Joshua V. Dillon, Ian Langmore, Dustin Tran, Eugene Brevdo, Srinivas Vasudevan, Dave Moore, Brian Patton, Alex Alemi, Matt Hoffman, and Rif A. Saurous. TensorFlow Distributions. *arXiv*, 2017. URL <http://arxiv.org/abs/1711.10604>.

- [21] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural Ordinary Differential Equations. *ArXiv e-prints*, June 2018. URL <https://arxiv.org/abs/1806.07366>.
- [22] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models. *ArXiv e-prints*, October 2018. URL <https://arxiv.org/abs/1810.01367>.
- [23] L. Zhang, W. E, and L. Wang. Monge-Ampère Flow for Generative Modeling. *ArXiv e-prints*, September 2018. URL <https://arxiv.org/abs/1809.10188>.
- [24] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on pure and applied mathematics*, 44(4):375–417, 1991.
- [25] Luis A Caffarelli, NSF-CBMS Conference on the Monge Ampère Equation: Applications to Geometry, Optimization, and Mario Milman. *Monge Ampere equation: applications to geometry and optimization*. American Mathematical Society, 1998.
- [26] Cédric Villani. *Topics in optimal transportation*. Number 58. American Mathematical Soc., 2003.
- [27] Kip S Thorne and Roger D Blandford. *Modern Classical Physics: Optics, Fluids, Plasmas, Elasticity, Relativity, and Statistical Physics*. Princeton University Press, 2017.
- [28] Luigi Ambrosio, Nicola Gigli, and Giuseppe Savaré. *Gradient flows: in metric spaces and in the space of probability measures*. Springer Science & Business Media, 2008.
- [29] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223, International Convention Centre, Sydney, Australia, 06–11 Aug 2017. PMLR. URL <http://proceedings.mlr.press/v70/arjovsky17a.html>.
- [30] Olivier Bousquet, Sylvain Gelly, Ilya Tolstikhin, Carl-Johann Simon-Gabriel, and Bernhard Schoelkopf. From optimal transport to generative modeling: the VEGAN cookbook. 2017. URL <http://arxiv.org/abs/1705.07642>.
- [31] Aude Genevay, Gabriel Peyré, and Marco Cuturi. GAN and VAE from an Optimal Transport Point of View. 2017. URL <http://arxiv.org/abs/1706.01807>.

- [32] Diederik P Kingma and Max Welling. Auto-Encoding Variational Bayes. 2013. URL <http://arxiv.org/abs/1312.6114>.
- [33] Peter Dayan, Geoffrey E Hinton, Radford M Neal, and Richard S Zemel. The Helmholtz Machine. 904:889–904, 1995.
- [34] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [35] Ronald J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Mach. Learn.*, 8:229–256, 1992. doi: 10.1023/A:1022672621406.
- [36] George Tucker, Andriy Mnih, Chris J. Maddison, Dieterich Lawson, and Jascha Sohl-Dickstein. REBAR: Low-variance, unbiased gradient estimates for discrete latent variable models. *arXiv*, 2017. URL <http://arxiv.org/abs/1703.07370>.
- [37] Song Cheng, Jing Chen, and Lei Wang. Information Perspective to Probabilistic Modeling: Boltzmann Machines versus Born Machines. *arXiv*, 2017. URL <http://arxiv.org/abs/1712.04144>.
- [38] Zhao-Yu Han, Jun Wang, Heng Fan, Lei Wang, and Pan Zhang. Unsupervised Generative Modeling Using Matrix Product States. 2017. URL <http://arxiv.org/abs/1709.01662>.
- [39] Ding Liu, Shi-Ju Ran, Peter Wittek, Cheng Peng, Raul Blázquez García, Gang Su, and Maciej Lewenstein. Machine Learning by Two-Dimensional Hierarchical Tensor Networks: A Quantum Information Theoretic Perspective on Deep Architectures. *arXiv*, 2017. URL <http://arxiv.org/abs/1710.04833>.
- [40] E. M. Stoudenmire. Learning Relevant Features of Data with Multi-scale Tensor Networks. 2017. URL <http://arxiv.org/abs/1801.00315>.
- [41] Raphael Bailly. Quadratic weighted automata:spectral algorithm and likelihood maximization. In Chun-Nan Hsu and Wee Sun Lee, editors, *Proceedings of the Asian Conference on Machine Learning*, volume 20 of *Proceedings of Machine Learning Research*, pages 147–163, South Garden Hotels and Resorts, Taoyuan, Taiwan, 14–15 Nov 2011. PMLR. URL <http://proceedings.mlr.press/v20/bailly11.html>.
- [42] Ming-Jie Zhao and Herbert Jaeger. Norm-observable operator models. *Neural Computation*, 22(7):1927–1959, 2010. doi: 10.1162/neco.2010.03-09-983. URL <https://doi.org/10.1162/neco.2010.03-09-983>. PMID: 20141473.
- [43] W. Huggins, P. Patel, K. B. Whaley, and E. Miles Stoudenmire. Towards Quantum Machine Learning with Tensor Networks. *ArXiv e-prints*, March 2018.

- [44] Yoav Levine, David Yakira, Nadav Cohen, and Amnon Shashua. Deep Learning and Quantum Entanglement: Fundamental Connections with Implications to Network Design. 2017. URL <http://arxiv.org/abs/1704.01552>.
- [45] A. Cichocki, A-H. Phan, Q. Zhao, N. Lee, I. V. Oseledets, M. Sugiyama, and D. Mandic. Tensor Networks for Dimensionality Reduction and Large-Scale Optimizations. Part 2 Applications and Future Perspectives. *arXiv*, 2017. doi: 10.1561/22000000067. URL <http://arxiv.org/abs/1708.09165>.
- [46] Shakir Mohamed and Balaji Lakshminarayanan. Learning in Implicit Generative Models. URL <http://arxiv.org/abs/1610.03483>.
- [47] Yujia Li, Kevin Swersky, and Richard Zemel. Generative moment matching networks. . URL <http://arxiv.org/abs/1502.02761>.
- [48] Gintare Karolina Dziugaite, Daniel M Roy, and Zoubin Ghahramani. Training generative neural networks via maximum mean discrepancy optimization. URL <https://arxiv.org/abs/1505.03906>.
- [49] N.H. Anderson, P. Hall, and D.M. Titterington. Two-sample test statistics for measuring discrepancies between two multivariate probability density functions using kernel-based density estimates. *Journal of Multivariate Analysis*, 50(1):41 – 54, 1994. ISSN 0047-259X. URL <http://www.sciencedirect.com/science/article/pii/S0047259X84710335>.
- [50] Arthur Gretton, Karsten M Borgwardt, Malte Rasch, Bernhard Schölkopf, and Alex J Smola. A kernel method for the two-sample-problem. In *Advances in Neural Information Processing Systems*, pages 513–520, 2007. URL <http://papers.nips.cc/paper/3110-a-kernel-method-for-the-two-sample-problem.pdf>.
- [51] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(Mar):723–773, 2012. URL <http://www.jmlr.org/papers/v13/gretton12a.html>.
- [52] Thomas Hofmann, Bernhard Schölkopf, and Alexander J Smola. Kernel methods in machine learning. *The annals of statistics*, pages 1171–1220, 2008. URL <http://www.jstor.org/stable/25464664>.
- [53] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models*. MIT Press, 2009.

- [54] Xi Chen, Diederik P. Kingma, Tim Salimans, Yan Duan, Prafulla Dhariwal, John Schulman, Ilya Sutskever, and Pieter Abbeel. Variational Lossy Autoencoder. *arXiv*, 2016. URL <http://arxiv.org/abs/1611.02731>.
- [55] Chun-Liang Li, Wei-Cheng Chang, Yu Cheng, Yiming Yang, and Barnabás Póczos. MMD GAN: Towards Deeper Understanding of Moment Matching Network. (Theorem 1), . URL <http://arxiv.org/abs/1705.08584>.
- [56] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355: 602, 2017.
- [57] G. Carleo, Y. Nomura, and M. Imada. Constructing exact representations of quantum many-body systems with deep neural networks. *ArXiv e-prints*, February 2018. URL <https://arxiv.org/abs/1802.09558>.
- [58] B. Jónsson, B. Bauer, and G. Carleo. Neural-network states for the classical simulation of quantum computing. *ArXiv e-prints*, August 2018. URL <https://arxiv.org/abs/1808.05232>.
- [59] Stephen R. Clark. Unifying Neural-network Quantum States and Correlator Product States via Tensor Networks. 2017. URL <http://arxiv.org/abs/1710.03545>.
- [60] Ivan Glasser, Nicola Pancotti, Moritz August, Ivan D. Rodriguez, and J. Ignacio Cirac. Neural Networks Quantum States, String-Bond States and chiral topological states. *Phys. Rev. X*, 8:11006, 2017. doi: 10.1103/PhysRevX.8.011006. URL <http://arxiv.org/abs/1710.04045>.
- [61] Dong Ling Deng, Xiaopeng Li, and S. Das Sarma. Quantum entanglement in neural network states. *Phys. Rev. X*, 7:1–17, 2017. doi: 10.1103/PhysRevX.7.021021.
- [62] Xun Gao and Lu-Ming Duan. Efficient Representation of Quantum Many-body States with Deep Neural Networks. *Nat. Commun.*, pages 1–5, 2017. doi: 10.1038/s41467-017-00705-2. URL <http://arxiv.org/abs/1701.05039><http://dx.doi.org/10.1038/s41467-017-00705-2>.
- [63] Raphael Kaubruegger, Lorenzo Pastori, and Jan Carl Budich. Chiral Topological Phases from Artificial Neural Networks. *arXiv*, 2017. URL <http://arxiv.org/abs/1710.04713>.
- [64] Zi Cai and Jinguo Liu. Approximating quantum many-body wave-functions using artificial neural networks. 035116:1–8, 2017. doi: 10.1103/PhysRevB.97.035116. URL <http://arxiv.org/abs/1704.05148>.

- [65] Kenny Choo, Giuseppe Carleo, Nicolas Regnault, and Titus Neupert. Symmetries and many-body excitations with neural-network quantum states. *Phys. Rev. Lett.*, 121:167204, Oct 2018. doi: 10.1103/PhysRevLett.121.167204. URL <https://link.aps.org/doi/10.1103/PhysRevLett.121.167204>.
- [66] J. Han, L. Zhang, H. Wang, and W. E. Solving Many-Electron Schro2dinger Equation Using Deep Neural Networks. *ArXiv e-prints*. URL <http://arxiv.org/abs/1807.07014>.
- [67] D. Wu, L. Wang, and P. Zhang. Solving Statistical Mechanics using Variational Autoregressive Networks. *ArXiv e-prints*, September 2018. URL <https://arxiv.org/abs/1809.10606>.
- [68] Cédric Bény. Deep learning and the renormalization group. *arXiv*, 2013. URL <http://arxiv.org/abs/1301.3124>.
- [69] Pankaj Mehta and David J. Schwab. An exact mapping between the Variational Renormalization Group and Deep Learning. *arXiv*, 2014. URL <http://arxiv.org/abs/1410.3831>.
- [70] Serena Bradde and William Bialek. PCA Meets RG. *J. Stat. Phys.*, 167:462–475, 2017. doi: 10.1007/s10955-017-1770-6.
- [71] Maciej Koch-Janusz and Zohar Ringel. Mutual Information, Neural Networks and the Renormalization Group. *arXiv*, 2017. URL <http://arxiv.org/abs/1704.06279>.
- [72] Shuo-Hui Li and Lei Wang. Neural Network Renormalization Group. *arXiv*, 2018. URL <http://arxiv.org/abs/1802.02840>.
- [73] Yi-Zhuang You, Zhao Yang, and Xiao-Liang Qi. Machine learning spatial geometry from entanglement features. *Phys. Rev. B*, 97:045153, Jan 2018. doi: 10.1103/PhysRevB.97.045153. URL <https://link.aps.org/doi/10.1103/PhysRevB.97.045153>.
- [74] C. E. Rasmussen. Gaussian Processes to Speed up Hybrid Monte Carlo for Expensive Bayesian Integrals. *Bayesian Stat.* 7, pages 651–659, 2003. URL http://www.is.tuebingen.mpg.de/fileadmin/user/_upload/files/publications/pdf2080.pdf.
- [75] Simon Duane, A D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid Monte Carlo. *Phys. Lett. B*, 195:216–222, 1987. doi: [https://doi.org/10.1016/0370-2693\(87\)91197-X](https://doi.org/10.1016/0370-2693(87)91197-X). URL <http://www.sciencedirect.com/science/article/pii/037026938791197X>.
- [76] Junwei Liu, Yang Qi, Zi Yang Meng, and Liang Fu. Self-learning Monte Carlo method. *Phys. Rev. B*, 95:1–5, 2017. doi: 10.1103/PhysRevB.95.041101.

- [77] Li Huang, Yi Feng Yang, and Lei Wang. Recommender engine for continuous-time quantum Monte Carlo methods. *Phys. Rev. E*, 95:031301(R), 2017. doi: 10.1103/PhysRevE.95.031301.
- [78] Li Huang and Lei Wang. Accelerated Monte Carlo simulations with restricted Boltzmann machines. *Phys. Rev. B*, 95:035105, 2017. doi: 10.1103/PhysRevB.95.035105.
- [79] Junwei Liu, Huitao Shen, Yang Qi, Zi Yang Meng, and Liang Fu. Self-learning Monte Carlo method and cumulative update in fermion systems. *Phys. Rev. B*, 95:241104, 2017. doi: 10.1103/PhysRevB.95.241104.
- [80] Faming Liang, Chuanhai Liu, and Raymond J Carroll. *Advanced Markov Chain Monte Carlo Methods: Learning from Past Samples*. Wiley, 2011.
- [81] Jiaming Song, Shengjia Zhao, and Stefano Ermon. A-NICE-MC: Adversarial Training for MCMC. *arXiv*, 2017. URL <http://arxiv.org/abs/1706.07561>.
- [82] Daniel Levy, Matthew D. Hoffman, and Jascha Sohl-Dickstein. Generalizing Hamiltonian Monte Carlo with Neural Networks. *arXiv*, 2017. URL <http://arxiv.org/abs/1711.09268>.
- [83] Marco F. Cusumano-Towner and Vikash K. Mansinghka. Using probabilistic programs as proposals. *arXiv*, 2018. URL <http://arxiv.org/abs/1801.03612>.
- [84] Troels Arnfred Bojesen. Policy Guided Monte Carlo: Reinforcement Learning Markov Chain Dynamics. *arXiv*, 2018. URL <http://arxiv.org/abs/1808.09095>.
- [85] Lei Wang. Can Boltzmann Machines Discover Cluster Updates ? *arXiv*, 2017. doi: 10.1103/PhysRevE.96.051301. URL <http://arxiv.org/abs/1702.08586>.
- [86] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *arXiv*, 2016. doi: 10.1021/acscentsci.7b00572. URL <http://arxiv.org/abs/1610.02415>.
- [87] Benjamin Sanchez-Lengeling and Alán Aspuru-Guzik. Inverse molecular design using machine learning: Generative models for matter engineering. *Science*, 361(6400):360–365, 2018.
- [88] Giacomo Torlai and Roger G. Melko. Neural Decoder for Topological Codes. *Phys. Rev. Lett.*, 119:030501, 2017. doi: 10.1103/PhysRevLett.119.030501.

- [89] Giacomo Torlai, Guglielmo Mazzola, Juan Carrasquilla, Matthias Troyer, Roger Melko, and Giuseppe Carleo. Many-body quantum state tomography with neural networks. *arXiv*, 2017. URL <http://arxiv.org/abs/1703.05334>.
- [90] J. Carrasquilla, G. Torlai, R. G. Melko, and L. Aolita. Reconstructing quantum states with generative models. *ArXiv e-prints*, October 2018. URL <http://arxiv.org/abs/1810.10584>.
- [91] Dong-Ling Deng. Machine learning detection of bell nonlocality in quantum many-body systems. *Phys. Rev. Lett.*, 120:240402, Jun 2018. doi: 10.1103/PhysRevLett.120.240402. URL <https://link.aps.org/doi/10.1103/PhysRevLett.120.240402>.
- [92] Scott Aaronson. Read the fine print. *Nat. Phys.*, 11:291–293, 2015. doi: 10.1038/nphys3272. URL <http://dx.doi.org/10.1038/nphys3272>.
- [93] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549:195–202, 2017. doi: 10.1038/nature23474. URL <http://dx.doi.org/10.1038/nature23474>.
- [94] Xun Gao, Zhengyu Zhang, and Luming Duan. An efficient quantum algorithm for generative machine learning. *arXiv*, 2017. URL <http://arxiv.org/abs/1711.02038>.
- [95] Marcello Benedetti, Delfina Garcia-Pintos, Yunseong Nam, and Alejandro Perdomo-Ortiz. A generative modeling approach for benchmarking and training shallow quantum circuits. *arXiv*, 2018. URL <http://arxiv.org/abs/1801.07686>.
- [96] H. Situ, Z. He, L. Li, and S. Zheng. Adversarial training of quantum Born machine. *ArXiv e-prints*, July 2018. URL <http://arxiv.org/abs/1807.01235>.
- [97] J. Zeng, Y. Wu, J.-G. Liu, L. Wang, and J. Hu. Learning and Inference on Generative Adversarial Quantum Circuits. *ArXiv e-prints*, August 2018. URL <https://arxiv.org/abs/1808.03425>.
- [98] J.-G. Liu and L. Wang. Differentiable Learning of Quantum Circuit Born Machine. *ArXiv e-prints*, April 2018. URL <https://arxiv.org/abs/1804.04168>.